Day 1 Exercise: JavaScript Basics

Topic: JavaScript Basics - Console Output, Naming Rules, Block Scope, Strict Mode
Estimated Time: 30-35 minutes
Difficulty: Beginner

Learning Goals

By completing this exercise, you will:

- Create and run JavaScript files with `node`
- Use `console.log()` to display text and values
- Store values in variables with good camelCase names
- Predict how `let` and `var` behave inside and outside a block
- Use `"use strict"` to catch a typo that JavaScript would otherwise hide
- Apply all of this to a banking scenario

Exercise Instructions

Part 1: Create Your Profile (5 min)

Create a new file called `day01-my-profile.js`. Store your profile details in variables, then print them. Use a banking job title as your role.

Requirements:

- Create one variable for each detail: name, role, years of experience, one banking tool you use, and why you're learning JavaScript
- Declare each variable with `let`
- Use camelCase names, for example `fullName` and `yearsOfExperience`
- Print each detail with a label in front, for example `"Name: " + fullName`

Example Output:

```
Name: Priya Sharma
Role: Loan Officer
Experience: 3 years
Tool: Core Banking System
Learning Goal: To automate repetitive account checks
```

Hints:

- Declare all your variables at the top of the file, then print them below
- Use `+` to join a label and a variable: `console.log("Role: " + jobRole);`

- Text values need quotes. Numbers don't: `let yearsOfExperience = 3;`

Part 2: Print a Transaction Report (10 min)

Create a new file called `day01-transaction-report.js` and write code to print a daily transaction report for a bank branch.

Requirements:

1. Print a report title (for example, "Daily Transaction Report")
2. Print 5 transactions, each with a transaction ID and an amount
3. Give each transaction a status: APPROVED, DECLINED, or PENDING. Use each status at least once
4. Use blank lines to make it readable
5. Print a summary at the end (for example, "3 Approved, 1 Declined, 1 Pending")

Example Output:

```
========================================
        DAILY TRANSACTION REPORT
========================================

Transaction 1: TXN-101, Amount: 5000
Status: APPROVED

Transaction 2: TXN-102, Amount: 12000
Status: DECLINED

Transaction 3: TXN-103, Amount: 800
Status: PENDING

Transaction 4: TXN-104, Amount: 25000
Status: APPROVED

Transaction 5: TXN-105, Amount: 3000
Status: APPROVED

========================================
        SUMMARY
========================================
Total Transactions: 5
Approved: 3
Declined: 1
Pending: 1
========================================
```

Hints:

- Use `console.log("")` for blank lines
- Use `console.log("==========")` for borders

Part 3: Print Customer Records (5 min)

Create a new file called `day01-customer-data.js` and write code to print details for 3 bank customers.

Requirements:

- Print 3 customers, each with an account ID, holder name, and balance
- Format it so each customer's details are easy to read on their own

Example Output:

```
CUSTOMER RECORDS
================

Customer 1:
Account ID: ACC-201
Holder Name: Alice Fernandes
Balance: 45000

Customer 2:
Account ID: ACC-202
Holder Name: Ben Miller
Balance: 12500

Customer 3:
Account ID: ACC-203
Holder Name: Chloe Davis
Balance: 78000

Done.
```

Hints:

- Use blank lines between customers for readability

Part 4: Predict, Then Run (5 min)

Create a new file called `day01-scope-predictions.js` and copy the four snippets below into it, in this order.

Before you run the file, write what you think each snippet will print as a comment above it. Then run the file and compare. If a prediction was wrong, write one line explaining why.

```
// Snippet A
let branchCode = "MUM-01";
if (true) {
  let branchCode = "PUN-02";
```

```
  console.log("Inside:", branchCode);
}
console.log("Outside:", branchCode);

// Snippet B
function showServiceFee() {
  console.log("Fee before:", serviceFee);
  var serviceFee = 250;
  console.log("Fee after:", serviceFee);
}
showServiceFee();

// Snippet C
function checkLimitWithVar() {
  if (true) {
    var dailyLimit = 50000;
  }
  console.log("Limit with var:", dailyLimit);
}
checkLimitWithVar();

// Snippet D
function checkLimitWithLet() {
  if (true) {
    let dailyLimit = 50000;
  }
  console.log("Limit with let:", dailyLimit);
}
checkLimitWithLet();
```

Hints:

- `console.log("Inside:", branchCode)` prints the label and the value with a space between them
- One snippet stops the program with an error. That's on purpose. Write down the error message you see
- Review `3-blockscope.js` from class if you get stuck

Part 5: Fix the Bugs (5 min)

Create a new file called `day01-fix-the-bugs.js` and copy this code into it exactly as it is:

```
"use strict";

let account-holder = "Meera Joshi";
let 2ndAccountId = "ACC-305";
let accountBalance = 15000;

acountBalance = 20000;
```

```
console.log("Holder:", account-holder);
console.log("Second account:", 2ndAccountId);
console.log("Balance:", accountBalance);
```

Requirements:

1. Run the file. Read the error, fix that line, and run it again. Repeat until it runs
2. Rename any invalid variable names to valid camelCase names. Update every line that uses them
3. Fix the typo so the balance is really updated to 20000
4. Once it works, put the typo back and delete the `"use strict";` line. Run it again. Write a comment at the top of the file explaining what changed and why that's dangerous
5. Put `"use strict";` back and fix the typo again before you submit

Example Output (after all fixes):

```
Holder: Meera Joshi
Second account: ACC-305
Balance: 20000
```

Hints:

- Node shows the line number of the problem. Start there
- Variable names can't contain hyphens and can't start with a number
- JavaScript is case-sensitive and spelling-sensitive. `acountBalance` and `accountBalance` are different names

---

Tricky Questions

Think through these before checking the answers key. They are the kind of questions that catch people out in interviews and code review.

1. If you declare `let balance` inside an `if` block, can you read it right after the block ends?
2. Are `accountType`, `AccountType`, and `accounttype` the same variable?
3. Without `"use strict"`, what happens if you assign a value to a variable you never declared?
4. Does `var` inside a `for` loop stay inside the loop once it finishes?

Bonus Challenges

For those who finish early and want extra practice:

Bonus 1: Add status symbols
In your transaction report, use plain text symbols instead of the status words, for example `[OK]` for approved, `[X]` for declined, and `[..]` for pending.

Bonus 2: Write an incident report
Print a detailed report for a failed transaction:

- Transaction ID
- Steps that led to the failure
- Expected result
- Actual result
- Severity

Bonus 3: Watch a loop counter leak
We cover loops properly on Day 6. For now, copy this code, run it, then change `var` to `let` and run it again. What changed, and why?

```
for (var i = 1; i <= 3; i++) {
  console.log("Processing transaction", i);
}
console.log("After loop:", i);
```

Reflection Questions

After completing the exercise, think about:

1. What was easiest about this exercise?
2. What was most challenging?
3. Which prediction in Part 4 surprised you, and why?
4. What questions do you have for the next class?

Write down your answers, we'll discuss in the next session.

Need Help?

Common Issues:

My code shows "SyntaxError"
Check that all text is inside quotes and console.log has parentheses:

```
// Wrong
console.log(Hello);

// Correct
console.log("Hello");
```

I see "ReferenceError: something is not defined"
Check the spelling and capital letters of the variable name. Also check that you're not using a `let` variable

outside the `{ }` block where you declared it.

Nothing prints when I run my code
Make sure you saved the file and are running the correct filename:

```
node day01-my-profile.js
```

I see "Cannot find module"
Check that your terminal is in the right folder before running `node`.

If you're still stuck, review the demo code from class or ask in the class chat.
